Amortized Efficiency for Pairing Delegation

Adrián P. Keilty

Chalmers University of Technology

March 19, 2026

Pairings

Let $(\mathbb{G}_1, +)$, $(\mathbb{G}_2, +)$ and $(\mathbb{G}_T, \cdot)$ be cyclic groups of order a 2λ-bit prime q where λ is the security parameter.

Pairings

Let $(\mathbb{G}_1, +)$, $(\mathbb{G}_2, +)$ and $(\mathbb{G}_T, \cdot)$ be cyclic groups of order a 2λ-bit prime q where λ is the security parameter. A cryptographic pairing is a map

$$e\colon \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T \quad \text{that is}$$

Pairings

Let $(\mathbb{G}_1, +)$, $(\mathbb{G}_2, +)$ and $(\mathbb{G}_T, \cdot)$ be cyclic groups of order a 2λ-bit prime q where λ is the security parameter. A cryptographic pairing is a map

$$e\colon \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T \quad \text{that is}$$

- Non degenerate: If $\langle P \rangle = \mathbb{G}_1$ and $\langle Q \rangle = \mathbb{G}_2$ then $e\langle e(P, Q) \rangle = \mathbb{G}_T$.

Pairings

Let $(\mathbb{G}_1, +)$, $(\mathbb{G}_2, +)$ and $(\mathbb{G}_T, \cdot)$ be cyclic groups of order a 2λ-bit prime q where λ is the security parameter. A cryptographic pairing is a map

$$e\colon \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T \quad \text{that is}$$

- Non degenerate: If $\langle P \rangle = \mathbb{G}_1$ and $\langle Q \rangle = \mathbb{G}_2$ then $e\langle e(P, Q) \rangle = \mathbb{G}_T$.
- Bilinear: $e([a]P, [b]Q) = e(P, Q)^{ab} = e([b]P, [a]Q)$.

Pairings

Let $(\mathbb{G}_1, +)$, $(\mathbb{G}_2, +)$ and $(\mathbb{G}_T, \cdot)$ be cyclic groups of order a 2λ-bit prime q where λ is the security parameter. A cryptographic pairing is a map

$$e\colon \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T \quad \text{that is}$$

- Non degenerate: If $\langle P \rangle = \mathbb{G}_1$ and $\langle Q \rangle = \mathbb{G}_2$ then $e\langle e(P, Q) \rangle = \mathbb{G}_T$.
- Bilinear: $e([a]P, [b]Q) = e(P, Q)^{ab} = e([b]P, [a]Q)$.
- Computable in polynomial time. In practice, instantiated over pairing-friendly curves such as BLS12-381.

Pairings

Let $(\mathbb{G}_1, +)$, $(\mathbb{G}_2, +)$ and $(\mathbb{G}_T, \cdot)$ be cyclic groups of order a 2λ-bit prime q where λ is the security parameter. A cryptographic pairing is a map

$$e\colon \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T \quad \text{that is}$$

- Non degenerate: If $\langle P \rangle = \mathbb{G}_1$ and $\langle Q \rangle = \mathbb{G}_2$ then $e\langle e(P, Q) \rangle = \mathbb{G}_T$.
- Bilinear: $e([a]P, [b]Q) = e(P, Q)^{ab} = e([b]P, [a]Q)$.
- Computable in polynomial time. In practice, instantiated over pairing-friendly curves such as BLS12-381.

Symmetric pairings ($\mathbb{G}_1 = \mathbb{G}_2$) <u>break</u> DDH on $\mathbb{G}_1$ and $\mathbb{G}_2$: given (g, g^a, g^b, g^c) one can test $c = ab$ via $e(g^a, g^b) \stackrel{?}{=} e(g, g^c)$.

Pairings

Let $(\mathbb{G}_1, +)$, $(\mathbb{G}_2, +)$ and $(\mathbb{G}_T, \cdot)$ be cyclic groups of order a 2λ-bit prime q where λ is the security parameter. A cryptographic pairing is a map

$$e\colon \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T \quad \text{that is}$$

- Non degenerate: If $\langle P \rangle = \mathbb{G}_1$ and $\langle Q \rangle = \mathbb{G}_2$ then $e\langle e(P, Q) \rangle = \mathbb{G}_T$.
- Bilinear: $e([a]P, [b]Q) = e(P, Q)^{ab} = e([b]P, [a]Q)$.
- Computable in polynomial time. In practice, instantiated over pairing-friendly curves such as BLS12-381.

Symmetric pairings ($\mathbb{G}_1 = \mathbb{G}_2$) <u>break</u> DDH on $\mathbb{G}_1$ and $\mathbb{G}_2$: given (g, g^a, g^b, g^c) one can test $c = ab$ via $e(g^a, g^b) \stackrel{?}{=} e(g, g^c)$.

- **New hardness assumption (DBDH)**: Given $(g_1, g_1^a, g_2, g_2^b, g_1^c, T)$ with $g_i \in \mathbb{G}_i$, hard to decide whether $T = e(g_1, g_2)^{abc}$ or random.

Pairing-based Cryptography

Pairing-based Cryptography

Joux's tripartite Diffie-Hellman: Alice broadcasts $[a]\,P$, Bob broadcasts $[b]\,P$, and Charlie broadcasts $[c]\,P$. Each then computes the shared key

$$K = e(P,P)^{abc} = \underset{\substack{\uparrow \\ \text{Alice}}}{e([b]P,[c]P)^{a}} = \underset{\substack{\uparrow \\ \text{Bob}}}{e([a]P,[c]P)^{b}} = \underset{\substack{\uparrow \\ \text{Charlie}}}{e([a]P,[b]P)^{c}}$$

Pairing-based Cryptography

Joux's tripartite Diffie-Hellman: Alice broadcasts $[a]P$, Bob broadcasts $[b]P$, and Charlie broadcasts $[c]P$. Each then computes the shared key

$$K = e(P,P)^{abc} = \underset{\substack{\uparrow \\ \text{Alice}}}{e([b]P,[c]P)^a} = \underset{\substack{\uparrow \\ \text{Bob}}}{e([a]P,[c]P)^b} = \underset{\substack{\uparrow \\ \text{Charlie}}}{e([a]P,[b]P)^c}$$

- **Short Signatures (BLS)**: Signatures are single $\mathbb{G}_1$ elements; aggregation of n signatures verified with a single pairing.

Pairing-based Cryptography

Joux's tripartite Diffie-Hellman: Alice broadcasts $[a]P$, Bob broadcasts $[b]P$, and Charlie broadcasts $[c]P$. Each then computes the shared key

$$K = e(P,P)^{abc} = \underset{\substack{\uparrow \\ \text{Alice}}}{e([b]P,[c]P)^a} = \underset{\substack{\uparrow \\ \text{Bob}}}{e([a]P,[c]P)^b} = \underset{\substack{\uparrow \\ \text{Charlie}}}{e([a]P,[b]P)^c}$$

- **Short Signatures (BLS)**: Signatures are single $\mathbb{G}_1$ elements; aggregation of n signatures verified with a single pairing.
- **Identity-Based Encryption (IBE)**: User's identity string serves as public key (Boneh-Franklin scheme).

Pairing-based Cryptography

Joux's tripartite Diffie-Hellman: Alice broadcasts $[a]P$, Bob broadcasts $[b]P$, and Charlie broadcasts $[c]P$. Each then computes the shared key

$$K = e(P,P)^{abc} = \underset{\substack{\uparrow \\ \text{Alice}}}{e([b]P,[c]P)^a} = \underset{\substack{\uparrow \\ \text{Bob}}}{e([a]P,[c]P)^b} = \underset{\substack{\uparrow \\ \text{Charlie}}}{e([a]P,[b]P)^c}$$

- **Short Signatures (BLS)**: Signatures are single $\mathbb{G}_1$ elements; aggregation of n signatures verified with a single pairing.
- **Identity-Based Encryption (IBE)**: User's identity string serves as public key (Boneh-Franklin scheme).
- **KZG Polynomial Commitments**: Constant-size commitments and opening proofs, foundational to modern zkSNARKs.

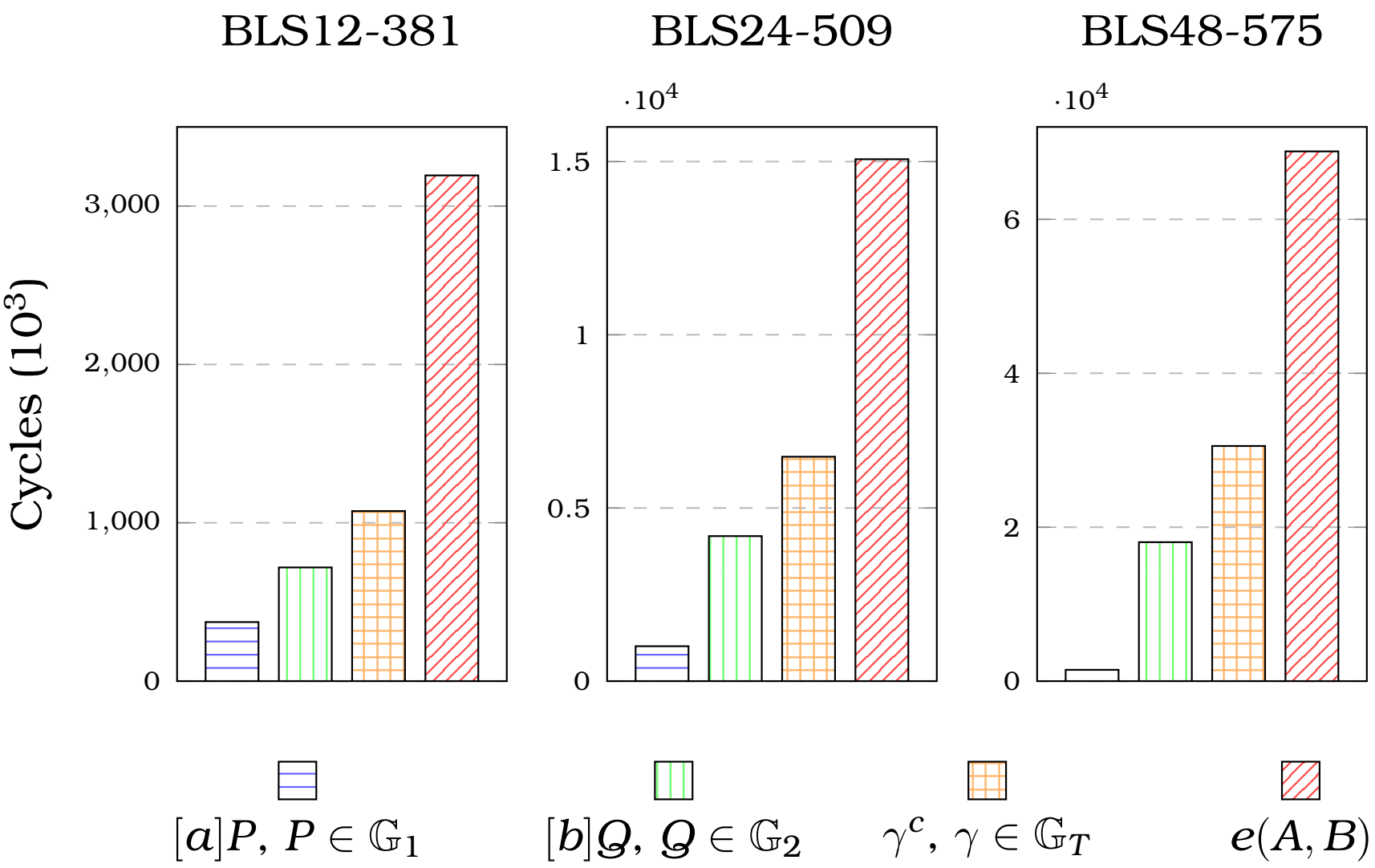
Pairing-based Cryptography

Joux's tripartite Diffie-Hellman: Alice broadcasts $[a]P$, Bob broadcasts $[b]P$, and Charlie broadcasts $[c]P$. Each then computes the shared key

$$K = e(P,P)^{abc} = \underset{\substack{\uparrow \\ \text{Alice}}}{e([b]P,[c]P)^a} = \underset{\substack{\uparrow \\ \text{Bob}}}{e([a]P,[c]P)^b} = \underset{\substack{\uparrow \\ \text{Charlie}}}{e([a]P,[b]P)^c}$$

- **Short Signatures (BLS)**: Signatures are single $\mathbb{G}_1$ elements; aggregation of n signatures verified with a single pairing.
- **Identity-Based Encryption (IBE)**: User's identity string serves as public key (Boneh-Franklin scheme).
- **KZG Polynomial Commitments**: Constant-size commitments and opening proofs, foundational to modern zkSNARKs.
- **zkSNARKs (Groth16)**: Constant-size zkSNARK proofs (3 group elements) with verification reducible to 3 pairings.

Computational Costs

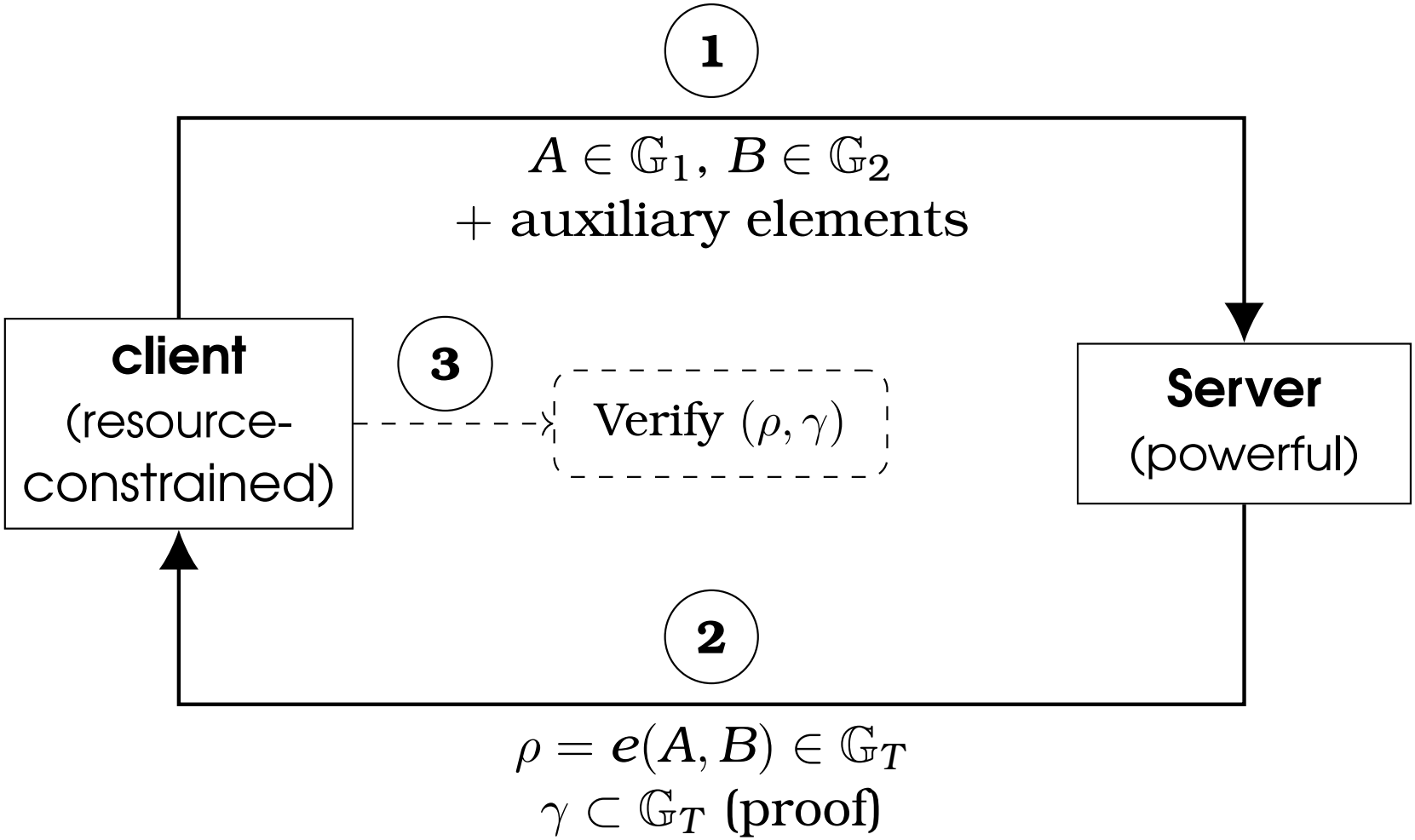


Pairing Delegation

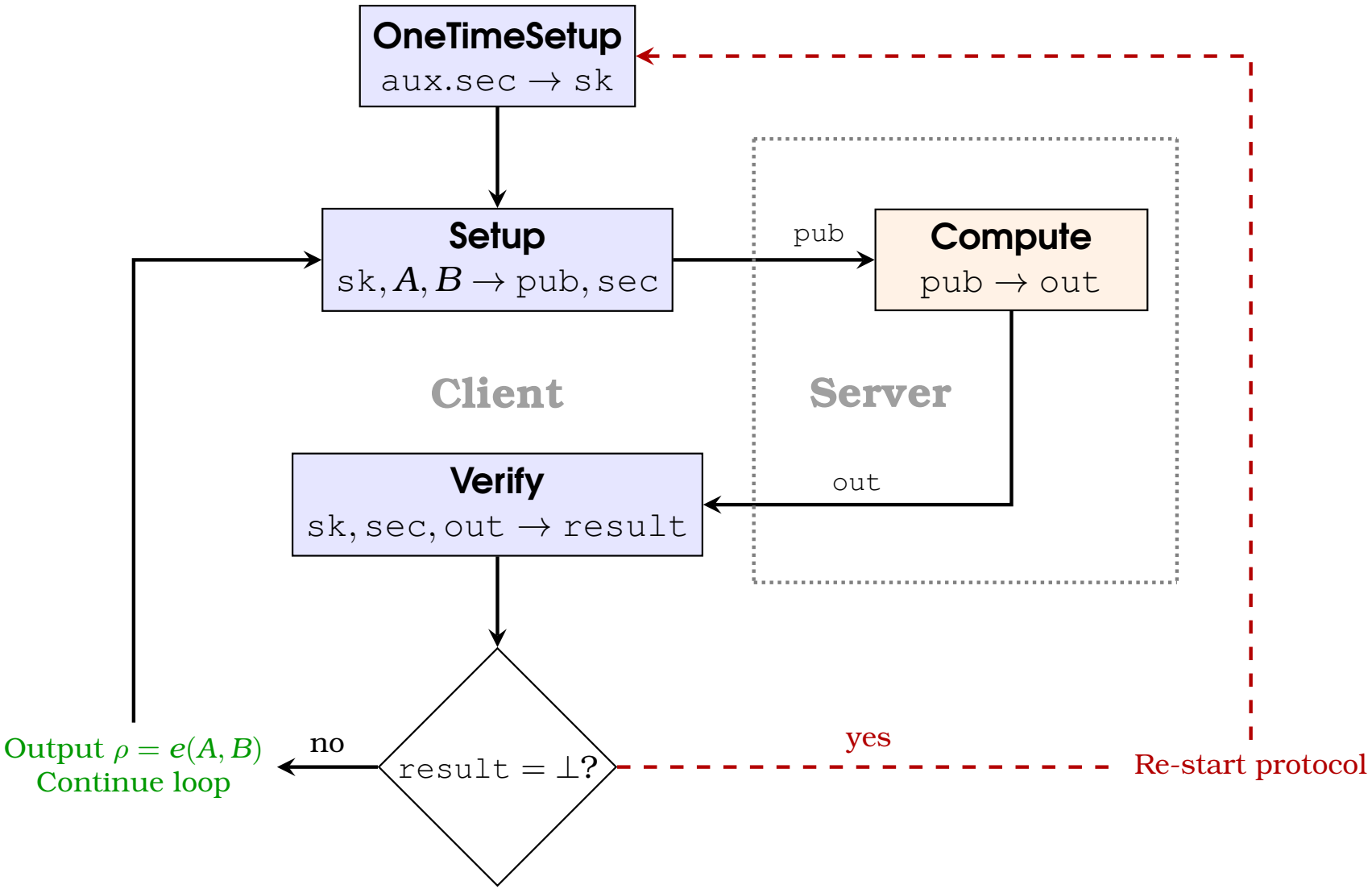
All pairing delegation protocols are

- One-shot:
- Proven unconditionally secure $\rightarrow$ This

One-Shot Pairing Delegation



Sequential Pairing Delegation



Delegation Properties

Delegation Properties

- **Correctness**: If the server behaves honestly, the output of the verification phase is the intended (vector of) pairing(s).

$$\Pr\left[\ \mathtt{result} = e(A,B)\ \middle|\ \begin{array}{c}(\mathtt{pub},\mathtt{sec}) \leftarrow \mathsf{Setup}(\mathtt{sk},A,B)\\ \mathtt{out} \leftarrow \mathsf{Compute}(\mathtt{pub})\\ \mathtt{result} \leftarrow \mathsf{Verify}(\mathtt{sk},\mathtt{sec},\mathtt{out})\end{array}\right] = 1\,.$$

Delegation Properties

- **Correctness**: If the server behaves honestly, the output of the verification phase is the intended (vector of) pairing(s).

$$\Pr\left[\ \mathtt{result} = e(A,B)\ \middle|\ \begin{array}{c}(\mathtt{pub},\mathtt{sec}) \leftarrow \mathsf{Setup}(\mathtt{sk},A,B)\\ \mathtt{out} \leftarrow \mathsf{Compute}(\mathtt{pub})\\ \mathtt{result} \leftarrow \mathsf{Verify}(\mathtt{sk},\mathtt{sec},\mathtt{out})\end{array}\right] = 1\,.$$

- **Verifiability**: The client must be able to detect with high probability when the server returns an incorrect result.

$$\Pr\left[\ \mathtt{out}^* \leftarrow \mathcal{A}\,(\mathtt{pub})\ \middle|\ \begin{array}{c}\mathtt{out}^* \neq \mathtt{out}\\ \mathsf{Verify}(\mathtt{sk},\mathtt{sec},\mathtt{out}) \neq \perp\end{array}\right] \leq \mathsf{negl}(\lambda)\,.$$

Delegation Properties

- **Correctness**: If the server behaves honestly, the output of the verification phase is the intended (vector of) pairing(s).

$$\Pr\left[\ \mathtt{result} = e(A,B)\ \middle|\ \begin{array}{c}(\mathtt{pub},\mathtt{sec}) \leftarrow \mathsf{Setup}(\mathtt{sk},A,B)\\ \mathtt{out} \leftarrow \mathsf{Compute}(\mathtt{pub})\\ \mathtt{result} \leftarrow \mathsf{Verify}(\mathtt{sk},\mathtt{sec},\mathtt{out})\end{array}\right] = 1\,.$$

- **Verifiability**: The client must be able to detect with high probability when the server returns an incorrect result.

$$\Pr\left[\ \mathtt{out}^* \leftarrow \mathcal{A}(\mathtt{pub})\ \middle|\ \begin{array}{c}\mathtt{out}^* \neq \mathtt{out}\\ \mathsf{Verify}(\mathtt{sk},\mathtt{sec},\mathtt{out}) \neq \perp\end{array}\right] \leq \mathsf{negl}(\lambda)\,.$$

- **Amortized Efficiency**: The client's workload when delegating L pairings must be lower than computing them locally.

$$\frac{\mathsf{cost}(\mathsf{OneTimeSetup}) + L \cdot (\mathsf{cost}(\mathsf{Setup}) + \mathsf{cost}(\mathsf{Verify}))}{L} \leq \mathsf{cost}(e(A,B))$$

One way to design this

One way to design this

For **correctness** and **verifiability**:

One way to design this

For **correctness** and **verifiability**:

Construct a minimal verification equation in $\mathbb{G}_T$ that relies on client generated secrets for it to be satisfied.

One way to design this

For **correctness** and **verifiability**:

Construct a minimal verification equation in $\mathbb{G}_T$ that relies on client generated secrets for it to be satisfied.

In particular, for public delegated inputs (A, B) and generators $(P, Q, \gamma_T = e(P, Q)) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$,

One way to design this

For **correctness** and **verifiability**:

Construct a minimal verification equation in $\mathbb{G}_T$ that relies on client generated secrets for it to be satisfied.

In particular, for public delegated inputs (A, B) and generators $(P, Q, \gamma_T = e(P, Q)) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$,

- generate secrets (u, v, r), and set $\xi = \gamma_T^{-u \cdot v} = e([u]P, [-v]Q) = e(U, -V)$,

One way to design this

For **correctness** and **verifiability**:

Construct a minimal verification equation in $\mathbb{G}_T$ that relies on client generated secrets for it to be satisfied.

In particular, for public delegated inputs (A, B) and generators $(P, Q, \gamma_T = e(P, Q)) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$,

- generate secrets (u, v, r), and set $\xi = \gamma_T^{-u \cdot v} = e([u]P, [-v]Q) = e(U, -V)$,
- send $\mathtt{pub} = (C, D) = ([v](U + A), V - [r]B)$,

One way to design this

For **correctness** and **verifiability**:

Construct a minimal verification equation in $\mathbb{G}_T$ that relies on client generated secrets for it to be satisfied.

In particular, for public delegated inputs (A, B) and generators $(P, Q, \gamma_T = e(P, Q)) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$,

- generate secrets (u, v, r), and set $\xi = \gamma_T^{-u \cdot v} = e([u]P, [-v]Q) = e(U, -V)$,
- send $\texttt{pub} = (C, D) = ([v](U + A), V - [r]B)$,
- request for $\texttt{out} = (\rho, \gamma) = (e(A, B), e(A, D) \cdot e(C, Q))$,

One way to design this

For **correctness** and **verifiability**:

Construct a minimal verification equation in $\mathbb{G}_T$ that relies on client generated secrets for it to be satisfied.

In particular, for public delegated inputs (A, B) and generators $(P, Q, \gamma_T = e(P, Q)) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$,

- generate secrets (u, v, r), and set $\xi = \gamma_T^{-u \cdot v} = e([u]P, [-v]Q) = e(U, -V)$,
- send $\texttt{pub} = (C, D) = ([v](U + A), V - [r]B)$,
- request for $\texttt{out} = (\rho, \gamma) = (e(A, B), e(A, D) \cdot e(C, Q))$,
- and check $\left(\xi \stackrel{?}{=} \rho^r \cdot \gamma\right) \wedge \left(\rho \stackrel{?}{\in} \mathbb{G}_T\right)$.

One way to design this

For **correctness** and **verifiability**:

Construct a minimal verification equation in $\mathbb{G}_T$ that relies on client generated secrets for it to be satisfied.

In particular, for public delegated inputs (A, B) and generators $(P, Q, \gamma_T = e(P, Q)) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$,

- generate secrets (u, v, r), and set $\xi = \gamma_T^{-u \cdot v} = e([u]P, [-v]Q) = e(U, -V)$,
- send $\texttt{pub} = (C, D) = ([v](U + A), V - [r]B)$,
- request for $\texttt{out} = (\rho, \gamma) = (e(A, B), e(A, D) \cdot e(C, Q))$,
- and check $\left(\xi \stackrel{?}{=} \rho^r \cdot \gamma\right) \wedge \left(\rho \stackrel{?}{\in} \mathbb{G}_T\right)$.

This guarantees unconditional security. Why?

One way to design this

Lemma 1 (Identifying forgeries)

Let $\mathcal{A}$ be an algebraic adversary, $(\xi, r) \in \mathbb{G}_T \times \mathbb{Z}_q^*$ the client's secret values, $\mathtt{out} = (\rho, \gamma) \in \mathbb{G}_T \times \mathbb{G}_T$ the output of an honest server, and Verify *a verification phase consisting of the checks*

$$\left(\xi \stackrel{?}{=} \rho^r \cdot \gamma\right) \wedge \left(\rho \stackrel{?}{\in} \mathbb{G}_T\right) . \tag{1}$$

If $(\mathcal{A}(\mathtt{pub}) \rightarrow \mathtt{out}^* \neq \mathtt{out}) \wedge (\mathsf{Verify}(\mathtt{out}^*) \neq \perp)$ then $\mathcal{A}$ outputs the secret r.

One way to design this

Lemma 1 (Identifying forgeries)

Let $\mathcal{A}$ be an algebraic adversary, $(\xi, r) \in \mathbb{G}_T \times \mathbb{Z}_q^*$ the client's secret values, $\mathtt{out} = (\rho, \gamma) \in \mathbb{G}_T \times \mathbb{G}_T$ the output of an honest server, and Verify *a verification phase consisting of the checks*

$$\left(\xi \stackrel{?}{=} \rho^r \cdot \gamma\right) \wedge \left(\rho \stackrel{?}{\in} \mathbb{G}_T\right) . \tag{1}$$

If $(\mathcal{A}(\mathtt{pub}) \to \mathtt{out}^* \neq \mathtt{out}) \wedge (\mathsf{Verify}(\mathtt{out}^*) \neq \perp)$ then $\mathcal{A}$ outputs the secret r.

Proof.

By the closure property of $\mathbb{G}_T$, if $\left(\xi = (\rho^*)^r \cdot (\gamma^*)\right) \wedge (\rho^* \in \mathbb{G}_T)$, then $\mathtt{out}^*$ can be written as a deviation from the honest output: $\rho^* = \rho \cdot \eta$ and $\gamma^* = \gamma \cdot \eta^{-r}$ for any public element $\eta \in \mathbb{G}_T$. Indeed, $(\rho^*)^r \cdot (\gamma^*) = (\rho \cdot \eta)^r \cdot \gamma \cdot \eta^{-r} = \rho^r \cdot \gamma = \xi$. □

One way to design this

For **amortized efficiency**:

One way to design this

For **amortized efficiency**:

- Sequential delegation: Reuse ξ by generating in the next delegations secrets (r_i, u_i, v_i) such that $u_i \cdot v_i = u_j \cdot v_j$. The public values will still be equipped with an extra degree of freedom.

One way to design this

For **amortized efficiency**:

- Sequential delegation: Reuse ξ by generating in the next delegations secrets (r_i, u_i, v_i) such that $u_i \cdot v_i = u_j \cdot v_j$. The public values will still be equipped with an extra degree of freedom.
- Sample short scalars. Short $\mathbb{G}_T$ exponentation is (a lot) cheaper than a full-range one.

One way to design this

For **amortized efficiency**:

- Sequential delegation: Reuse ξ by generating in the next delegations secrets (r_i, u_i, v_i) such that $u_i \cdot v_i = u_j \cdot v_j$. The public values will still be equipped with an extra degree of freedom.
- Sample short scalars. Short $\mathbb{G}_T$ exponentation is (a lot) cheaper than a full-range one.

Wait a minute... do we still get unconditional security?

Meet-in-the-middle attack

A toy example:

Let $(\mathbb{G} = \langle P \rangle, +)$ be a cyclic group of prime order q, and (r_1, r_2, u) a tuple of secret values such that $r_1, r_2 \xleftarrow{\$} S \subset \mathbb{Z}_q^*$ and $\varepsilon \xleftarrow{\$} \mathbb{Z}_q^*$. Let $\xi = [\varepsilon]P \in \mathbb{G}$, and consider the public view

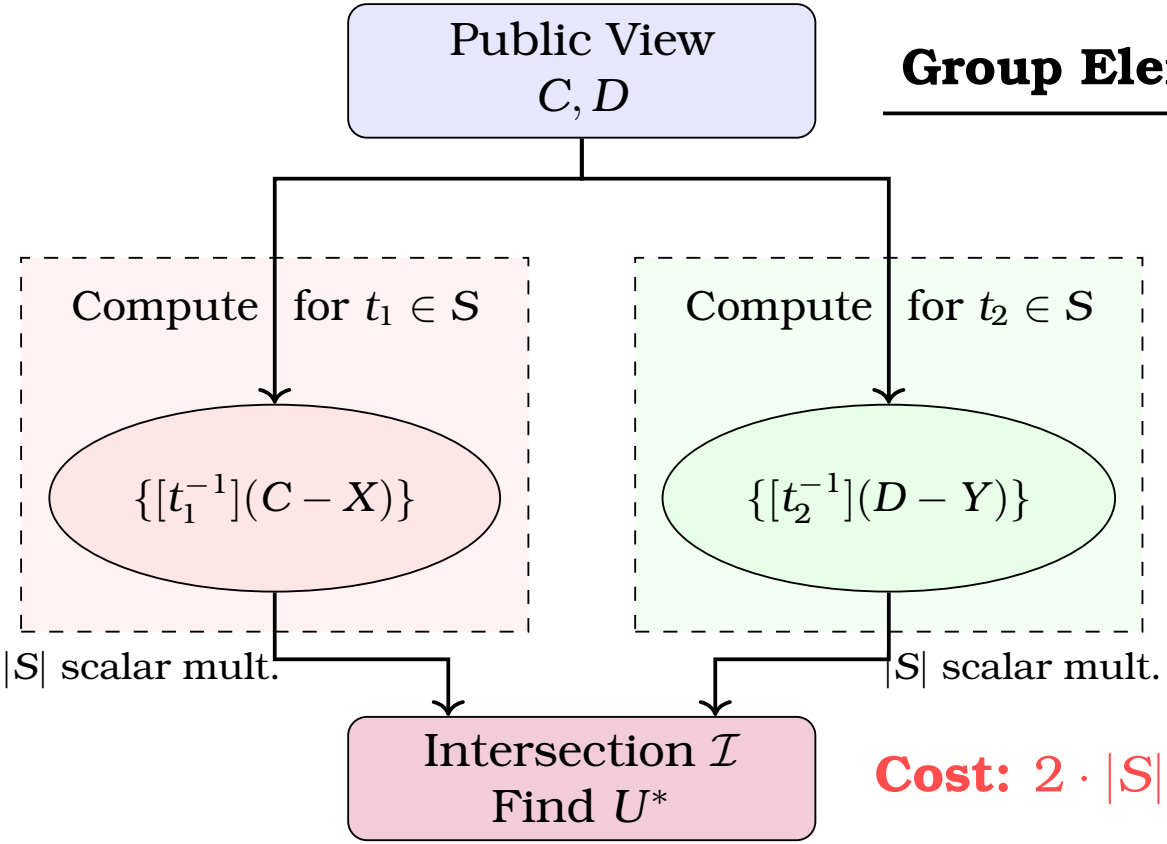
$$\mathtt{pub} = \begin{cases} C = [r_1]\xi + X \\ D = [r_2]\xi + Y \end{cases}, \tag{2}$$

where $X, Y \in \mathbb{G}$ are public and $X \neq C, Y \neq D$. $\mathcal{A}$ can attempt to learn the secret tuple from (2) by picking an element

$$\xi^* \in \mathcal{I} = \left\{[t_1^{-1}](C - X) \colon t_1 \in S\right\} \cap \left\{[t_2^{-1}](D - Y) \colon t_2 \in S\right\} \tag{3}$$

and its corresponding indexes r_1^*, r_2^*, which takes up to $2 \cdot |S|$ computations.

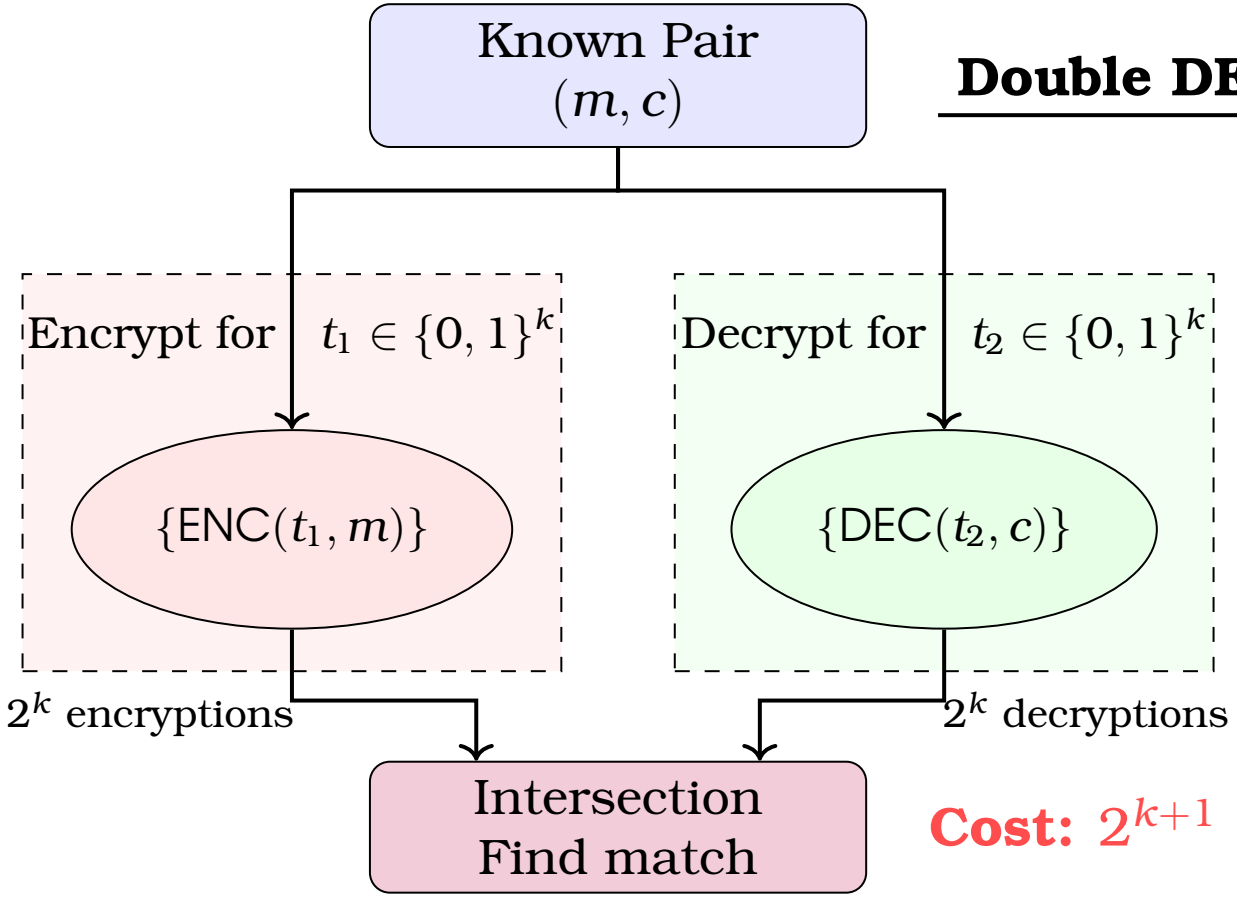
Meet-in-the-middle attack



Group Element Intersection Attack

Cost: $2 \cdot |S|$ **scalar multiplications**

Meet-in-the-middle attack



Double DES Meet-in-the-Middle

Cost: 2^{k+1} operations

What now?

What now?

If short scalars are not used, minimal efficiency is gained. If short scalars are used, no unconditional security can be proven...

What now?

If short scalars are not used, minimal efficiency is gained. If short scalars are used, no unconditional security can be proven...

Remark: Once the protocol execution is finished, $\mathcal{A}$ can no longer cheat. We could aim for everlasting security instead.

What now?

If short scalars are not used, minimal efficiency is gained. If short scalars are used, no unconditional security can be proven...

Remark: Once the protocol execution is finished, $\mathcal{A}$ can no longer cheat. We could aim for everlasting security instead.

(Unorthodox) Solution:

What now?

If short scalars are not used, minimal efficiency is gained. If short scalars are used, no unconditional security can be proven...

Remark: Once the protocol execution is finished, $\mathcal{A}$ can no longer cheat. We could aim for everlasting security instead.

(Unorthodox) Solution:

- Add a timeout parameter to the protocol.

What now?

If short scalars are not used, minimal efficiency is gained. If short scalars are used, no unconditional security can be proven...

Remark: Once the protocol execution is finished, $\mathcal{A}$ can no longer cheat. We could aim for everlasting security instead.

(Unorthodox) Solution:

- Add a timeout parameter to the protocol.
- Bound $\mathcal{A}$ computationally... but not in the conventional way: Assume $\mathcal{A}$ is not more powerful than the entire Bitcoin network, i.e., $\mathcal{A}$ is allowed to compute 2^κ scalar group multiplications per second (κ matching the current hashrate conversion).

Adversary winning probability

Adversary winning probability

Time-bound: $\mathcal{A}$ computes $\leq 2^{\kappa}$ short exponentations in τ seconds.

Adversary winning probability

Time-bound: $\mathcal{A}$ computes $\leq 2^{\kappa}$ short exponentations in τ seconds.
Best strategy: Choose $S_1, S_2 \subset [\![2^{\varphi}]\!] : |S_1| + |S_2| \leq 2^{\kappa}$ and intersect sets.

Adversary winning probability

Time-bound: $\mathcal{A}$ computes $\leq 2^{\kappa}$ short exponentations in τ seconds.
Best strategy: Choose $S_1, S_2 \subset [\![2^{\varphi}]\!] : \ |S_1| + |S_2| \leq 2^{\kappa}$ and intersect sets.

$$\Pr\left[\ \mathtt{out}^* \leftarrow \mathcal{A}(\mathtt{pub}) \ \middle|\ \begin{array}{c} \mathtt{out}^* \neq \mathtt{out} \\ \mathsf{Verify}(\mathtt{sk}, \mathtt{sec}, \mathtt{out}) \neq \perp \end{array}\right]$$

$$\leq \Pr\left[\xi \in \begin{array}{c} \text{set } \{[r^{-1}](C - X) : \ r \in S_1\} \\ \cap \\ \text{set } \{[r^{-1}](D - Y) : \ r \in S_2\} \end{array}\right]$$

$$\leq \Pr\left[\begin{array}{c} \text{event } \{r_1 \in S_1\} \\ \wedge \\ \text{event } \{r_2 \in S_2\} \end{array}\right] \leq 2^{-\sigma} \quad \text{if } \varphi = \lceil \frac{\sigma - 1}{2} + \kappa \rceil$$

Adversary winning probability

Time-bound: $\mathcal{A}$ computes $\leq 2^{\kappa}$ short exponentations in τ seconds.
Best strategy: Choose $S_1, S_2 \subset [\![2^{\varphi}]\!] : \ |S_1| + |S_2| \leq 2^{\kappa}$ and intersect sets.

$$\Pr\left[\ \mathtt{out}^* \leftarrow \mathcal{A}(\mathtt{pub}) \ \middle|\ \begin{array}{c} \mathtt{out}^* \neq \mathtt{out} \\ \mathsf{Verify}(\mathtt{sk}, \mathtt{sec}, \mathtt{out}) \neq \perp \end{array}\right]$$

$$\leq \Pr\left[\begin{array}{c} \text{event } \{r_1 \in S_1\} \\ \wedge \\ \text{event } \{r_2 \in S_2\} \end{array}\right] \leq 2^{-\sigma} \quad \text{if } \varphi = \lceil \frac{\sigma - 1}{2} + \kappa \rceil$$

Adversary winning probability

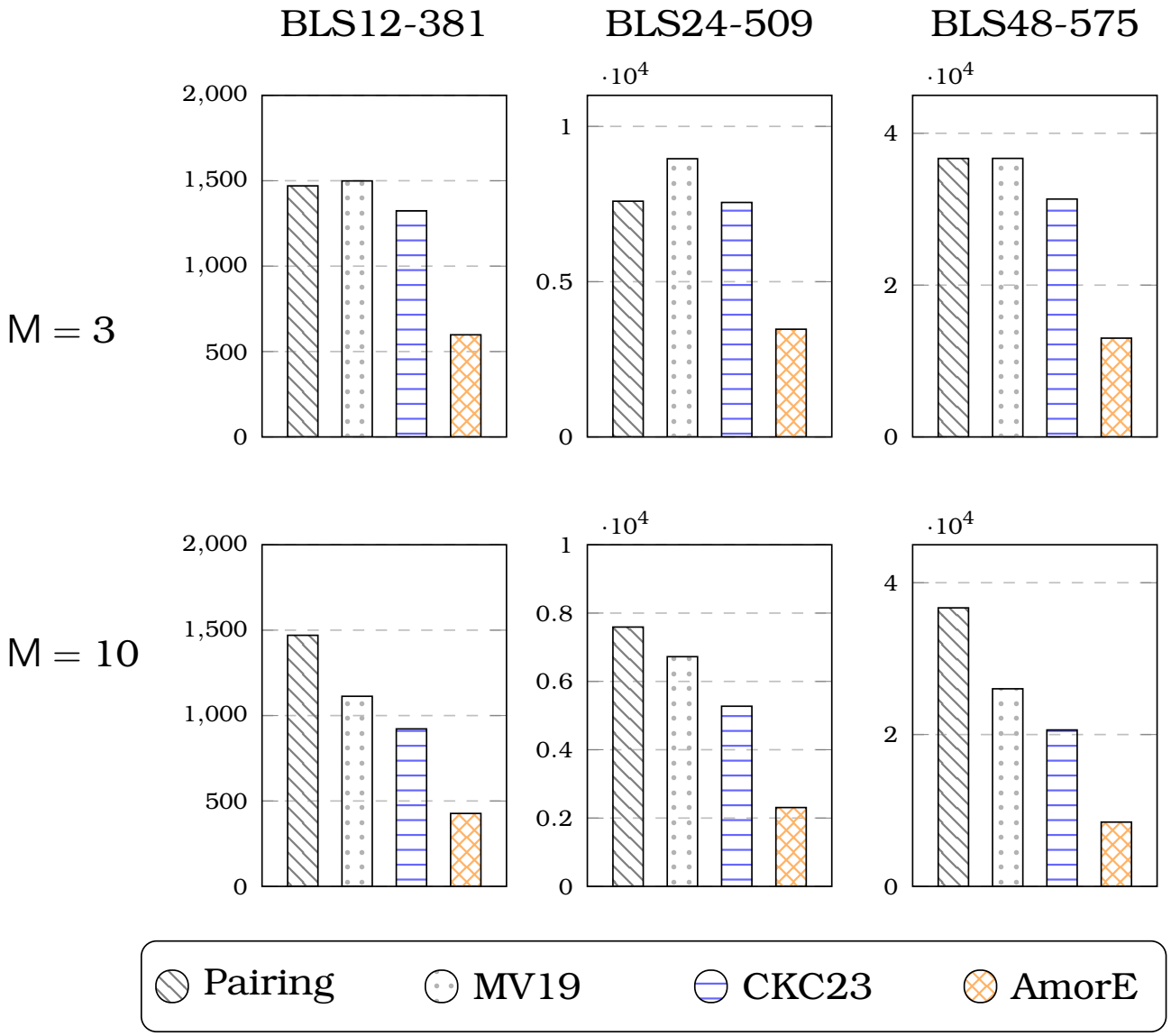
Time-bound: $\mathcal{A}$ computes $\leq 2^{\kappa}$ short exponentations in τ seconds.
Best strategy: Choose $S_1, S_2 \subset [\![2^{\varphi}]\!] : \ |S_1| + |S_2| \leq 2^{\kappa}$ and intersect sets.

$$\Pr\left[\ \mathtt{out}^* \leftarrow \mathcal{A}(\mathtt{pub}) \ \middle|\ \begin{array}{c} \mathtt{out}^* \neq \mathtt{out} \\ \mathsf{Verify}(\mathtt{sk}, \mathtt{sec}, \mathtt{out}) \neq \perp \end{array}\right]$$

$$\leq \Pr\left[\xi \in \begin{array}{c} \text{set } \{[r^{-1}](C - X) : \ r \in S_1\} \\ \cap \\ \text{set } \{[r^{-1}](D - Y) : \ r \in S_2\} \end{array}\right]$$

$$\leq \Pr\left[\begin{array}{c} \text{event } \{r_1 \in S_1\} \\ \wedge \\ \text{event } \{r_2 \in S_2\} \end{array}\right] \leq 2^{-\sigma} \quad \text{if } \varphi = \lceil \frac{\sigma - 1}{2} + \kappa \rceil$$

Performance results



Thank you for your attention :)

Paper: eprint.iacr.org/2025/542